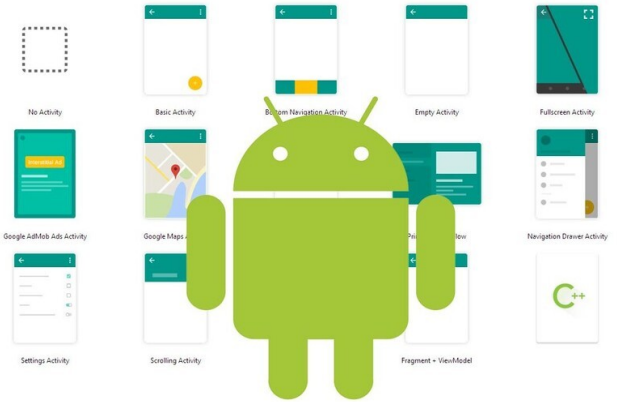
I. E. S.
Infanta Elena
Jumilla



UT 3
Las actividades en Android

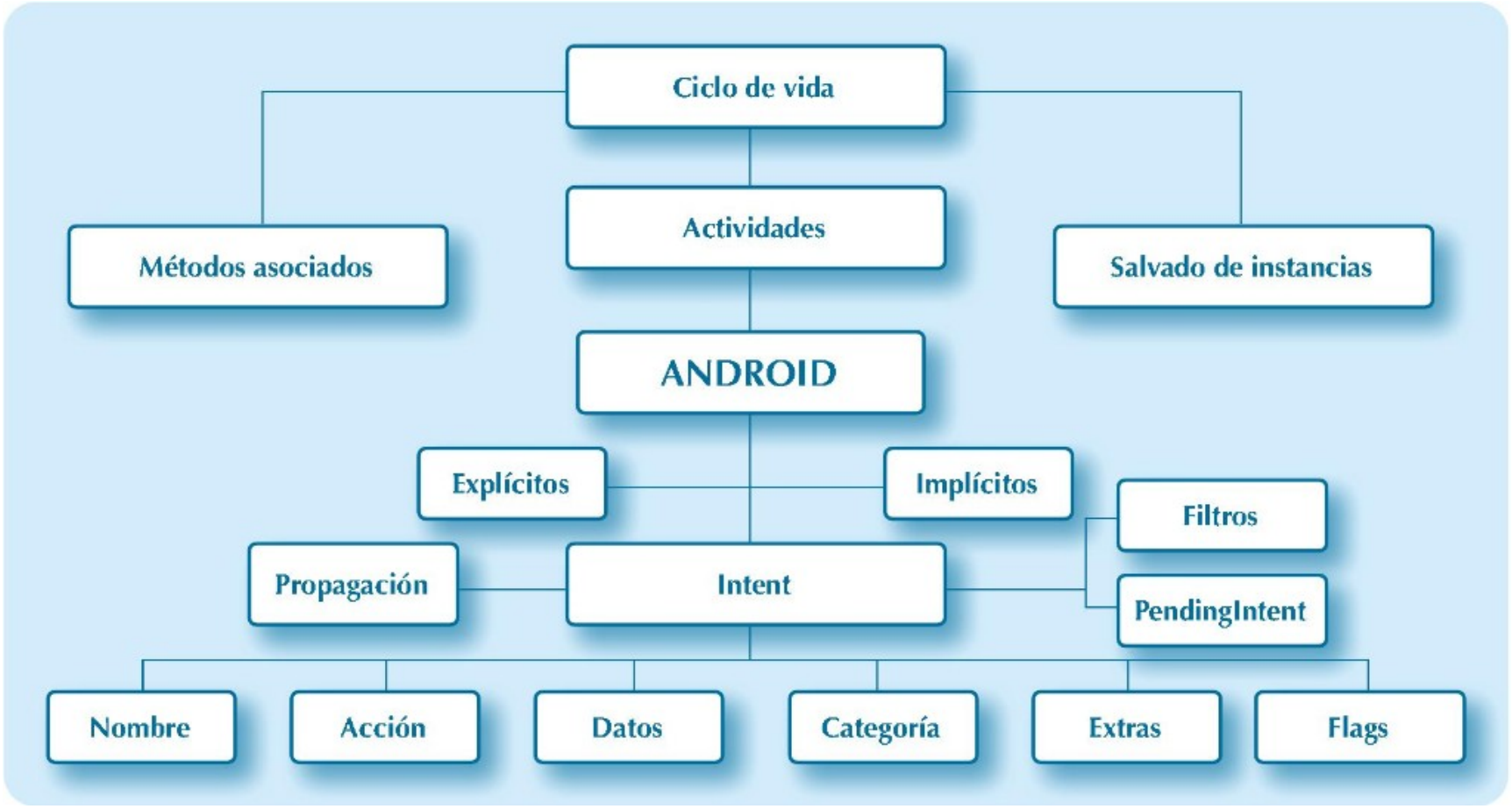
2º DAM - PMDM

Profesor: **Daniel Vicente Fernández**

Índice

- Definición
- Las actividades (*Activity*)
- Pila de actividades (*Back Stack*)
- Tareas
- Procesos
- Ciclo de vida de una actividad
- Hola Mundo
- Conservar el estado de una aplicación
- Intents

Mapa conceptual



Glosario

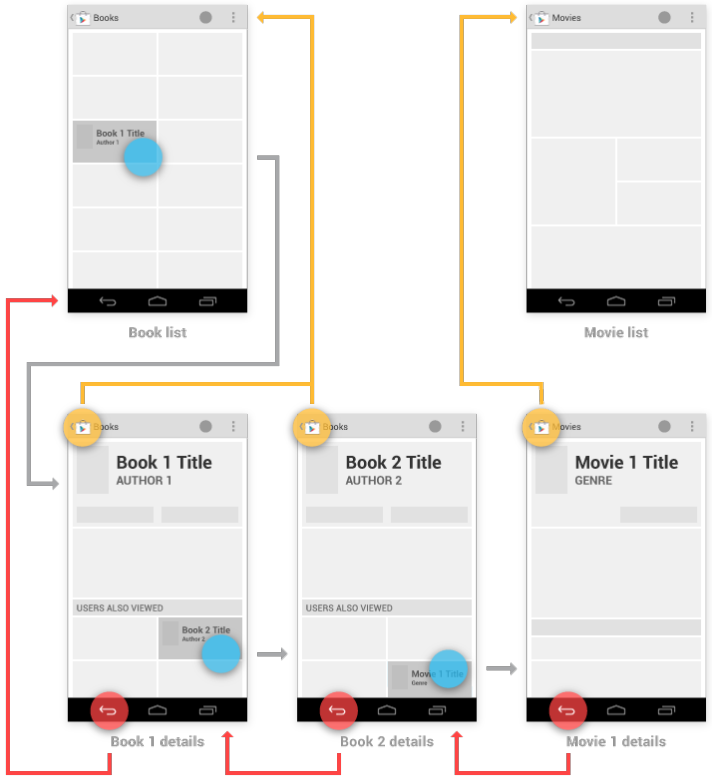
- ***API (Application Programming Interface)***. Conjunto de protocolos, funciones y comandos informáticos que permiten a los desarrolladores crear programas específicos para determinados sistemas operativos y, además, integrar el software de las aplicaciones.
- ***Callback* (llamada de vuelta)**. Es una función que recibe parámetros de una función primaria, que se ejecutará una vez que esta haya terminado de ejecutarse, realizando algún tipo de actividad según la información recibida.
- ***Foreground***. En términos informáticos, es contrario a Background, y hace alusión a aquellos procesos que se ejecutan en primer plano o que están mostrándose a través de la pantalla.
- ***Layout***. Define la estructura visual en la interfaz de usuario; actúa como contenedor de objetos. Puede declararse en Android mediante XML o añadiendo una instancia en Java en tiempo de ejecución.
- **Lenguaje de marcas**. Son aquellos lenguajes o formas de codificación en los que, junto al texto y el código, incorporan etiquetas o marcas que dan información suplementaria sobre la estructura del texto o su presentación.
- ***Override***. Término utilizado en Java (como lenguaje orientado a objetos) que permite sobrecargar o modificar el comportamiento original de un método heredado.
- ***URI (Uniform Resource Identifier)***. Es un estándar definido que permite identificar y facilitar la localización de recursos en Internet (páginas web, imágenes, vídeos...).

Definición

- Una **Activity** en Android es uno de los componentes fundamentales en el desarrollo de aplicaciones móviles.
- Representa **una única pantalla** con una interfaz de usuario.
- Es el punto de interacción principal entre el usuario y la aplicación.
- Una actividad hereda de la **clase Activity**.
- En una aplicación habrá tantas *activities* como pantallas distintas tenga la aplicación.
- Por ejemplo, pantalla de inicio o una pantalla de configuración.

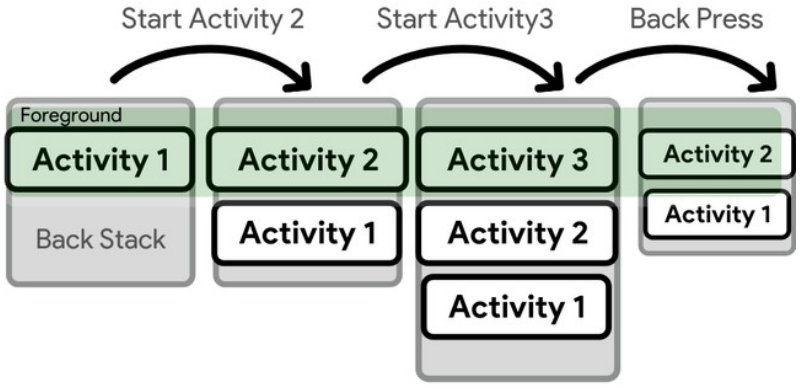
Las Actividades

- Una aplicación está formada por varias *activities* vinculadas de forma flexible.
- La secuencia de aparición de cada *activity* dependerá de cómo se haya programado el flujo del programa y de la interacción con el usuario.
- Android almacena la posición de cada actividad en una pila (LIFO) llamada ***Stack*** o ***Back Stack***.



Pila de actividades (*Stack* o *Back Stack*)

- Android almacena la posición de cada actividad en una pila (LIFO) llamada ***Stack*** o ***Back Stack***.
- Cada vez que una actividad se inicia obtiene el foco: posición superior de la pila.
- La actividad anterior se almacena en la pila en estado detenido.
- Cuando el usuario regresa a una actividad anterior, la que tenía el foco se elimina (sale de la pila) y la actividad inferior en la pila se restaura y obtiene el foco.



Para saber más

Tareas (*Tasks*)

- Una **tarea** (***task***) es un conjunto de actividades con las que el usuario interactúa para realizar un determinado trabajo.
- Las actividades de una tarea tienen una relación lógica y están almacenadas en la pila.
- Cuando todas las actividades desaparecen de la pila, la tarea deja de existir y desaparece de memoria.
- Una tarea pasa a **segundo plano** cuando el usuario comienza una nueva tarea o si va a la pantalla de inicio.
- Las actividades de una tarea en segundo plano se paran pero el *back stack* se conserva para poder restaurarlas cuando la tarea vuelva a primer plano.
- En caso de tener que **liberar memoria** las actividades pueden ser eliminadas pero su posición en la pila y otras variables deben ser almacenadas para poder restaurar la actividad a su estado antes de la eliminación (*onSaveInstanceState()*).

Procesos

- Las actividades funcionan dentro del dispositivo como procesos.
- Estados de un proceso:
  - ***Foreground Process***: proceso en el que se encuentra la actividad que se está mostrando en pantalla y con la que interactúa el usuario.
  - ***Visible Process***: actividad visible pero con la que no se puede interactuar. Estado muy útil para los programadores.
  - ***Service Process***: proceso con una actividad que funciona como servicio.
  - ***Background Process***: proceso con una actividad no visible y en un nivel inferior dentro de la pila.
  - ***Empty Process***: procesos vacíos que Android utiliza para crear nuevos procesos.

Para saber más

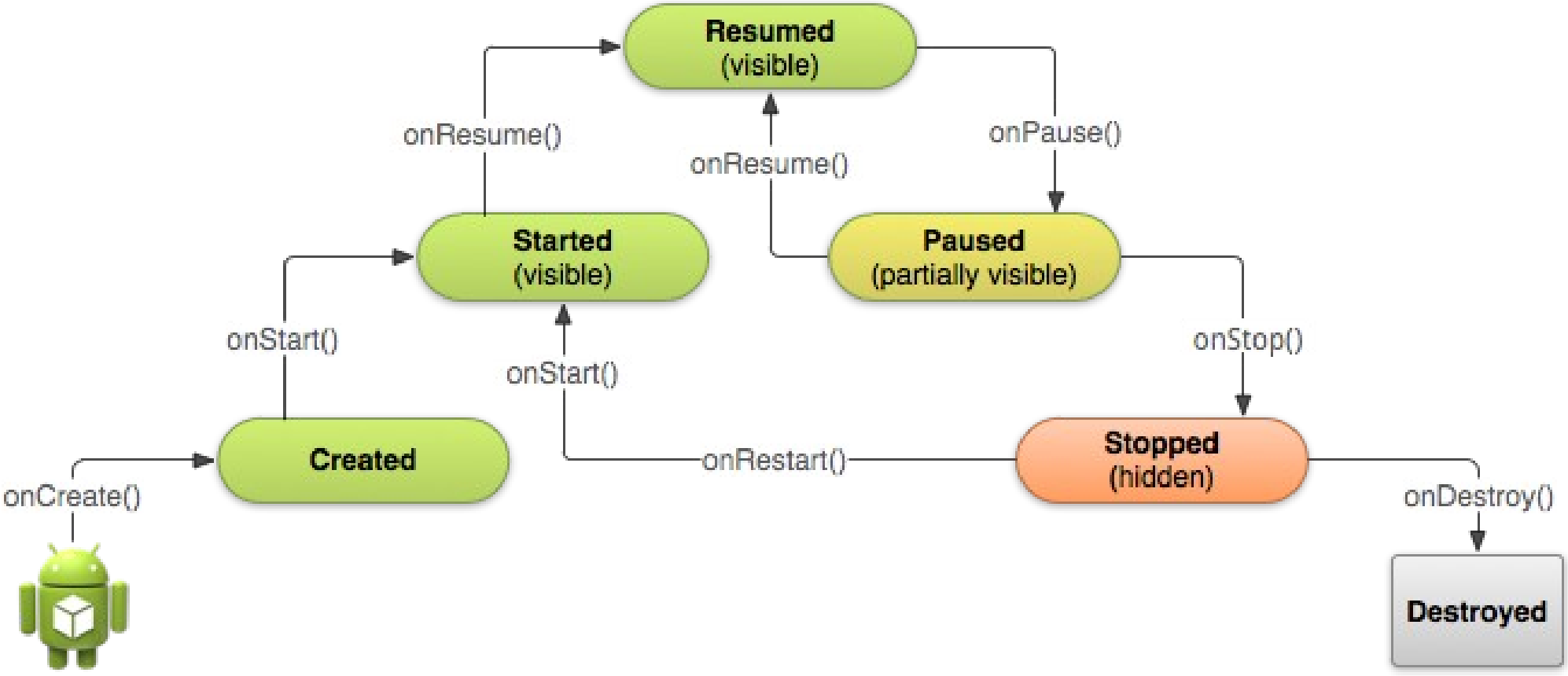
Ciclo de vida de una actividad

- Teniendo en cuenta el estado de funcionamiento de una actividad, esta puede ser:
  - **Inexistente**: no ocupa memoria y no se puede interactuar con ella.
  - **Detenida**: no es visible ni accesible pero si ocupa memoria.
  - **Pausada**: visible pero no se puede interactuar con ella.
  - **Activa**: está en primer plano y se puede interactuar con ella.

Ciclo de vida de una actividad II

- Métodos (*callback*) para controlar el ciclo de vida de una aplicación:
  - ***onCreate()***: Inicializa la actividad. Aquí se pueden configurar los componentes de la UI y cargar datos necesarios.
  - ***onStart()***: La actividad está a punto de ser visible. Permite ubicar instrucciones asociadas a la UI.
  - ***onResume()***: Cuando la actividad comienza a interactuar con el usuario.
  - ***onPause()***: Cuando otra actividad está a punto de tomar el foco. Se detienen procesos, animaciones, etc. Es parcialmente visible.
  - ***onStop()***: La actividad ya no es visible. La actividad sigue en memoria pero se reducen los recursos que consume.
  - ***onRestart()***: La actividad parada vuelve a primer plano. Le sigue un *onStart()*.
  - ***onDestroy()***: La actividad se está cerrando. Se liberan recursos y se elimina de memoria.

Ciclo de vida de una actividad III



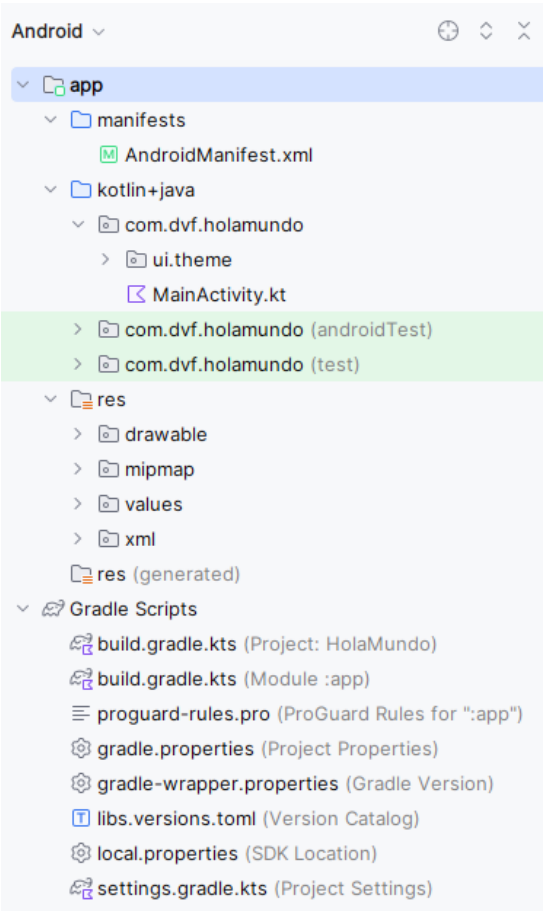
¡Hola mundo! de Android



- Crear proyecto nuevo: HolaMundo
- Dispositivo: teléfono móvil
- Tipo de actividad principal: actividad vacía.
- Paquete: *com.dvf.holamundo*
- SDK mínima: una superior al 95% de aceptación (Nougat).
- Lenguaje: Kotlin

¡Hola mundo! de Android II





MainActivity.kt | AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="HolaMundo"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.HolaMundo"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="HolaMundo"
            android:theme="@style/Theme.HolaMundo">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
```

AndroidManifest.xml

¡Hola mundo! de Android III




```
package com.dvf.holamundo

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import com.dvf.holamundo.ui.theme.HolaMundoTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            HolaMundoTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
```

Up-to-date

GreetingPreview

Hello Android!

MainActivity

- **Import:**
  - Activity (Pantalla)
  - Bundle (Almacenamiento)
- Override:
  - onCreate

Conservar el estado de la aplicación

¿Cómo conservar el estado de una aplicación cuando la actividad cambia de estado (el usuario rota la pantalla, navega a otra aplicación, recibe un mensaje)?

- ***Bundles***: En el método *onSaveInstanceState()*, se pueden almacenar datos que se recuperarán cuando la actividad se recree (por ejemplo, al rotar el dispositivo).
- ***DataStore***: Se utilizan para guardar configuraciones simples o valores pequeños de forma persistente.
- **Bases de datos SQLite** o **archivos**: Para almacenar datos más complejos que deben persistir incluso después de que la aplicación sea cerrada.

Intents

Un ***Intent*** permite a las actividades y componentes de una aplicación comunicarse entre sí o con otras aplicaciones.

Permiten reutilizar componentes que ya desarrollan una determinada funcionalidad sin necesidad de volver a implementerla.

Los Intents se utilizan fundamentalmente para:

- Iniciar otras actividades
- Iniciar servicios.
- Enviar información (*broadcast*) a otras partes del sistema operativo o aplicaciones externas.

Intents explícitos

Los Intents **explícitos** permitien iniciar una actividad específica dentro de tu aplicación o en otra aplicación.

- El uso habitual es para iniciar un componente en tu propia aplicación, porque conoces el nombre de clase de la actividad o el servicio que quieres iniciar.
- Se define explícitamente la clase de destino.
- Para ejemplo, puedes iniciar una nueva actividad dentro de la aplicación en respuesta a una acción del usuario, o iniciar un servicio que descargue un archivo en segundo plano.
- Parámetros: es necesario indicar el **contexto** y la **actividad** que se va a lanzar.
- **Importante**: todas las actividades que se ejecutan de manera independiente deben declararse en el *Manifest*.

```
// Executed in an Activity, so 'this' is the Context
// The fileUrl is a string URL, such as "http://www.example.com/image.png"
val downloadIntent = Intent(this, DownloadService::class.java).apply {
    data = Uri.parse(fileUrl)
}
startService(downloadIntent)
```

Intents implícitos

Los Intents **implícitos** permiten invocar componentes que pueden manejar acciones específicas, sin definir explícitamente el componente de destino.

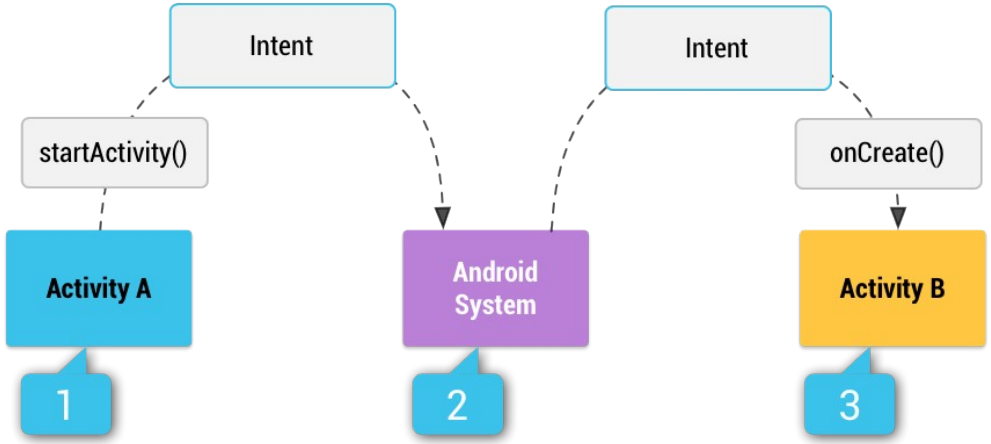
- El sistema Android busca el componente apropiado para manejar la acción en función de la información que se le ha pasado al Intent.
- Para ello se utilizan los filtros de Intents
- Por ejemplo, abrir un enlace web, enviar un correo electrónico, enviar una ubicación a un usuario para que la vea en una app de mapas.

```
// Create the text message with a string.
val sendIntent = Intent().apply {
    action = Intent.ACTION_SEND
    putExtra(Intent.EXTRA_TEXT, textMessage)
    type = "text/plain"
}

// Try to invoke the intent.
try {
    startActivity(sendIntent)
} catch (e: ActivityNotFoundException) {
    // Define what your app should do if no activity can handle the intent.
}
```

Intents implícitos II

- La actividad A crea un Intent con una descripción de la acción que desea realizar y se lo pasa al método *startActivity()*.
- El sistema Android busca en todas las aplicaciones un filtro de intents que coincida con con el Intent creado por la actividad A.
- Cuando el sistema encuentra una coincidencia ejecuta la actividad B (*onCreate()*) pasándole el Intent.



Elementos de un Intent

En un Intent podemos definir varios elementos para facilitar al sistema la selección del componente que debe realizar la tarea deseada.

- **Nombre del componente** (explícitos): hace explícito el componente que queremos que realice la tarea. (*setComponent(), setClass(), setClassName()*)
- **Acción**: determina la acción genérica a realizar. Podemos definir nuestras propias acciones o utilizar las definidas en la clase Intent *(ACTION_VIEW, ACTION_SEND, etc.*)
- **Datos**: URI con los datos que necesita el componente para trabajar. En ocasiones también es necesario definir el tipo de dato para "facilitarle" la tarea al sistema.
- **Categorías**(implícitos): complementa a la acción suministrando información sobre el tipo de componente necesario (*CATEGORY_BROWSABLE, CATEGORY_LAUNCHER, etc.*)
- **Extras**: información adicional para el componente lanzado. Muy útil en los explícitos para enviar información entre actividades.
- **Flags**: metadatos para que el sistema sepa cómo inicar una actividad y el comportamiento del componente que atienda la petición.

Para profundizar

Filtros de Intents

- Los IntentFilters Determinan la acción que puede llevar a cabo un componente y los tipos de Intents que pueden guestionarla.
- Deben definirse en el Manifest de las aplicaciones.
- Si hay varias coincidencias el sistema lanza un dialogo de selección.

```xml
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapp">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">

        <!-- Declarar la actividad -->
        <activity android:name=".ShareActivity">
            <intent-filter>
                <action android:name="android.intent.action.SEND" />

                <!-- El tipo de datos que la actividad puede manejar -->
                <category android:name="android.intent.category.DEFAULT" />
                <data android:mimeType="text/plain" />
                <data android:mimeType="image/*" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

AndroidManifest.xml

```kotlin
class ShareActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_share)

        // Obtener el intent que lanzó la actividad
        val intent = intent
        val action = intent.action
        val type = intent.type

        if (Intent.ACTION_SEND == action && type != null) {
            when {
                type.startsWith("text/") -> handleSendText(intent) // Manejar el texto compartido
                type.startsWith("image/") -> handleSendImage(intent) // Manejar la imagen compartida
            }
        }
    }

    private fun handleSendText(intent: Intent) {
        intent.getStringExtra(Intent.EXTRA_TEXT)?.let { sharedText ->
            // Haz algo con el texto compartido
            Log.d("ShareActivity", "Texto recibido: $sharedText")
        }
    }

    private fun handleSendImage(intent: Intent) {
        (intent.getParcelableExtra<Uri>(Intent.EXTRA_STREAM))?.let { imageUri ->
            // Haz algo con la imagen compartida
            Log.d("ShareActivity", "Imagen recibida: $imageUri")
        }
    }
}
```

Actividad que maneja el Intent

Navigation Compose

- Alternativa al uso de intents en metodologías más modernas de desarrollo como Jetpack Compose.
- **Single Activity**: una sola actividad (*MainActivity*) y se navega entre composables, lo cual es más eficiente y simplifica la gestión del ciclo de vida.

MainActivity.kt

```kotlin
// Clase principal de la app
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // Establece el contenido de la UI usando Jetpack Compose
        setContent {
            // Crea una instancia del controlador de navegación
            val navController = rememberNavController()

            // Define el grafo de navegación con sus rutas
            NavHost(navController, startDestination = "pantalla1") {
                // Ruta para Pantalla1
                composable("pantalla1") { Pantalla1(navController) }

                // Ruta para Pantalla2
                composable("pantalla2") { Pantalla2() }
            }
        }
    }
}
```

```kotlin
// Composable que representa la primera pantalla
// El parámetro NavController es necesario para
// poder pasar a pantalla2
@Composable
fun Pantalla1(navController: NavController) {
    // Botón que navega a Pantalla2 al hacer clic
    Button(onClick = {
        navController.navigate("pantalla2")
    }) {
        Text("Ir a Pantalla 2")
    }
}

// Composable que representa la segunda pantalla
@Composable
fun Pantalla2() {
    // Caja centrada en pantalla con un texto
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {
        Text(text = "Hola, estás en la siguiente pantalla")
    }
}
```

¡¡Fin!!